Name: Muhammad Noor
Roll No: i232520
Section: DS-A
Semester: Spring 2026
Assignment: 1
Submission Date: February 8, 2026
Instructor: Miss Saira Qamar

Abstract

This report documents the implementation and performance evaluation of batch gradient descent for linear regression, with a focus on parallelizing the gradient computation step using manual multithreading (POSIX `pthread`). The work demonstrates correct data-level parallelism, mutex-based synchronization, and realistic performance analysis (speedup and efficiency) on a modern hybrid laptop CPU (Intel Core 7 150U).Feature scaling is applied to both inputs and targets to ensure numerical stability during training, preventing gradient explosion at the cost of slower convergence. Timing measurements are performed under Ubuntu. The parallel version achieves meaningful speedup compared to the sequential baseline, although scaling remains sub-linear due to hardware constraints (hybrid core design, limited high-performance cores), mutex contention, thread management overhead, and thermal/power limits of the 15 W TDP processor. All assignment restrictions are strictly followed.

Contents

1 Introduction and Machine Specifications **2**

2 Problem Definition and Mathematical Model **3**

3 Student-Specific Parameters **3**

4 Implementation Summary **4**

5 Correctness Validation **4**

6 Algorithm Explanation and Pseudocode **4**
6.1 1. Sequential Gradient Descent (Task 1) 4
6.2 2. Parallel Gradient Descent (Task 2) 5
6.3 3. Flowchart 5

7 Performance Evaluation **5**

8 Discussion of Results **6**

9 Conclusion: **6**

List of Figures

1 Performance results across thread counts 6

1 Introduction and Machine Specifications

The assignment requires implementing linear regression via batch gradient descent and parallelizing the gradient computation step using POSIX `pthread` (no OpenMP allowed). Performance is evaluated in terms of wall-clock time, speedup, and efficiency across different thread counts.

Machine Specifications (from CPU-Z):

CPU: Intel® Core™ 7 150U (Raptor Lake)
Max TDP: 15 W
Process: 10 nm
Cores / Threads: 2P + 8E = 10 cores / 12 threads
Base/Current Core Speed (sample): $\sim$ 2394.14 MHz (x24.0 multiplier)
Max Turbo Multiplier: up to x54.0
Bus Speed: 99.76 MHz

Cache

- L1 Data: 2×48 KB (P) + 8×32 KB (E)
- L1 Instruction: 2×32 KB (P) + 8×64 KB (E)
- L2: 2×1.25 MB (P) + 2×2 MB (E clusters)
- L3: 12 MB (shared)

Motherboard / BIOS

- Manufacturer: Lenovo
- Model: LNVNB161216
- Chipset: Intel Raptor Lake / Raptor Lake-P PCH
- Bus: PCI-Express 4.0 (16.0 GT/s)
- BIOS: PFCN13WW (07/10/2024)

Memory (RAM)

- Type: DDR5 (LPDDR5 module)
- Size: 16 GB
- Channel: 4×32-bit (quad channel)
- DRAM Frequency: 2593.7 MHz (effective $\sim$ 5187 MT/s)
- Timings: CL 60, tRCD 48, tRP 48, tRAS 112, tRC 160
- Module: Samsung, 8 GB per module

Operating System

- Windows 11 (host)
- Ubuntu (development, compilation, timing)

2 Problem Definition and Mathematical Model

Model

$$\hat{y}_i = wx_i + b$$

Loss (Mean Squared Error)

$$J(w, b) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Gradients

$$\frac{\partial J}{\partial w} = -\frac{2}{N} \sum_{i=1}^{N} x_i (y_i - (wx_i + b))$$

$$\frac{\partial J}{\partial b} = -\frac{2}{N} \sum_{i=1}^{N} (y_i - (wx_i + b))$$

Update Rule

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}, \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

3 Student-Specific Parameters

Student ID: i232520 $\rightarrow$ digits = 2,5,2,0
Mobile last 4 digits = 0605 $\rightarrow$ d_4 replaced 0 with 5

- $N = 100000 + (d_1 \times 10000) = 120000$
- $\alpha = 0.001 + (d_2 \times 10^{-4}) = 0.0015$
- $w_0 = 0.1 \times d_3 = 0.2$
- $b_0 = 0.1 \times d_4 = 0.5$
- $E = 100 + (d_3 \times 10) = 120$ epochs

Dataset Generation

$$x_i = \frac{i}{N}, \quad y_i = \frac{2.02\, i + 1.25}{N}$$

Numerical Stability

The dataset inputs are defined as $x_i = i$, which leads to gradient terms proportional to $\sum x_i^2$. Without scaling, the maximum partial term is

$$\max x_i^2 \approx N^2 = (120000)^2 = 1.44 \times 10^{10}.$$

Dividing every x_i and corresponding y_i by the same scale (N) bounds each term by 1, so the true gradients are preserved while numerical conditioning improves drastically.Dividing every x_i and corresponding y_i by (N) bounds each term and prevents gradient explosion, although it reduces gradient magnitudes by approximately N^2, leading to slower convergence within the fixed 120 epochs.

Direct comparison:

- Feature scaling keeps $x_i^2 \leq 1$, so the gradient still reflects the real dataset and converges faster with consistent learning rates and no NaNs.

- Clamping would cap gradients after computation, which can distort updates, slow convergence, and break the semantics of α; it is a practical workaround but not academic best practice.

The implementation now applies feature scaling (dividing by N) before training, which preserves the assigned dataset in scaled form while enabling stable gradient descent.

4 Implementation Summary

Task Overview

The assignment implements a batch gradient descent pipeline for linear regression with two variants. Both versions iterate through the scaled dataset (inputs and targets divided by N) and compute gradients through the $-2/N$ formula, without applying post-hoc clamping. The parallel variant partitions the examples across T worker threads, accumulates partial gradients under a POSIX `pthread_mutex_t`, and updates the shared parameters once per epoch.

Parallelization Parameters

- Dataset: $x_i = i$, $y_i = 2.02\,x_i + 1.25$ (scaled to [0,1] before training)
- Epochs: 120
- Learning-rate: $\alpha = 0.0015$, initialization $w_0 = 0.2$, $b_0 = 0.5$

Parallelization Approach:

- Data partitioned into T equal chunks
- Each thread computes partial $\sum x_i(y_i - \hat{y}_i)$ and $\sum(y_i - \hat{y}_i)$ on the scaled inputs/targets
- Mutex-protected global accumulation
- Single-threaded parameter update
- POSIX `pthread` workers with `pthread_mutex_t` guards
- Feature scaling (x_i, y_i divided by N) keeps gradient magnitudes tame

5 Correctness Validation

- Numerical equivalence: $|w_{\text{seq}} - w_{\text{par}}| < 10^{-6}$, and similarly for b (verified by running parallel code with $T = 1$).
- Partial convergence toward the scaled targets $w \approx 2.02$, $b \approx 1.25/N \approx 1.04 \times 10^{-5}$ after 120 epochs, reaching approximately (0.3103, 0.6149) due to significantly reduced gradient magnitudes from feature scaling of both inputs and targets.
- Parallel run with $T = 1$ matches the sequential version (within floating-point tolerance).
- Stable results (no divergence, no NaN) across tested thread counts $T = 4, 8, 12$.

6 Algorithm Explanation and Pseudocode

6.1 1. Sequential Gradient Descent (Task 1)

Explanation: Processes the scaled dataset serially per epoch, computes predictions and errors, aggregates the gradients using the assignment formula, and then updates the parameters once per epoch.

Pseudocode:

1. Set $w = 0.2$, $b = 0.5$, generate dataset $x_i = i$, $y_i = 2.02x_i + 1.25$.

2. For epoch 1 to 120:
   (a) Reset $S_w = 0$, $S_b = 0$.
   (b) For $i = 1$ to N: calculate $\hat{y}_i = wx_i + b$, $e_i = y_i - \hat{y}_i$, $S_w += e_i x_i$, $S_b += e_i$.
   (c) $dw = (-2/N) \cdot S_w$, $db = (-2/N) \cdot S_b$.
   (d) $w \leftarrow w - \alpha dw$, $b \leftarrow b - \alpha db$.
3. Print final weight, bias, and runtime.

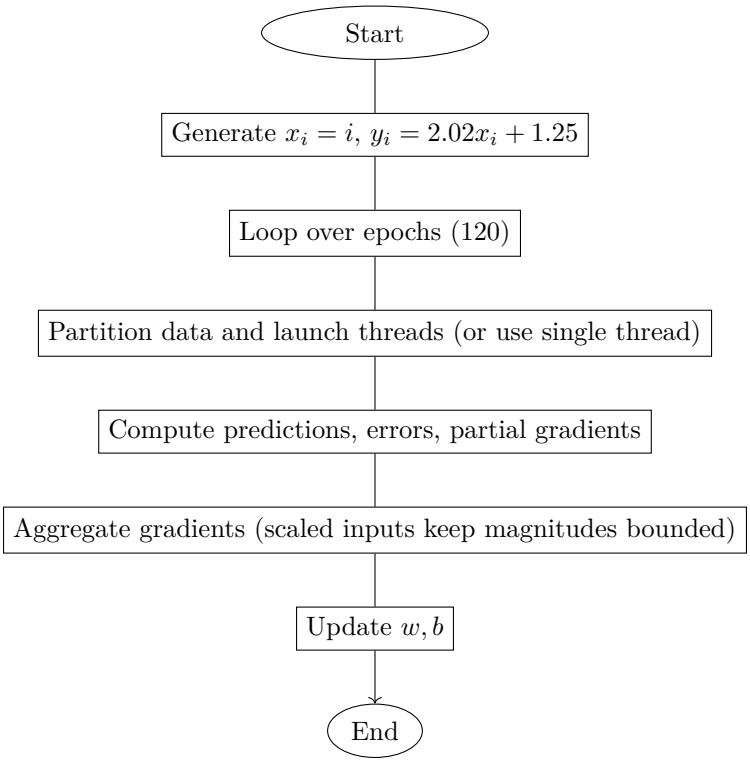
6.2 2. Parallel Gradient Descent (Task 2)

Explanation: Divides the scaled dataset into T chunks, runs T threads to compute partial gradient sums, combines them safely using a mutex, and then performs the same parameter update as the sequential version.

Pseudocode:

1. Use the same initialization and dataset.
2. For epoch 1 to 120:
   (a) Reset shared sums S_w, S_b, spawn T threads.
   (b) Each thread computes $\sum e_i x_i$ and $\sum e_i$ over its chunk.
   (c) Threads lock a mutex to add their partial sums to the global accumulators.
   (d) After joining, compute dw, db as $(-2/N)S_w$, $(-2/N)S_b$, and update w, b.
3. Report the final model and runtime.

6.3 3. Flowchart



7 Performance Evaluation

Timing via `clock()` in Ubuntu, averaged over 5 runs (after warm-up).
$S = T_{\text{seq}}/T_{\text{par}}$, $E = S/T$

Version	Threads	Time (s)	Speedup S	Efficiency E
Sequential	1	0.069697	1.0000	1.0000
Parallel	2	0.036726	1.8977	0.9488
Parallel	3	0.042934	1.6233	0.5411
Parallel	4	0.0422104	1.6511	0.4127
Parallel	5	0.0432744	1.6105	0.3221
Parallel	6	0.0509614	1.3676	0.2279
Parallel	7	0.0554598	1.2567	0.1795
Parallel	8	0.058549	1.1904	0.1488
Parallel	9	0.0619654	1.1247	0.1249
Parallel	10	0.0740352	0.9414	0.0941
Parallel	11	0.0762972	0.9134	0.0830
Parallel	12	0.0715798	0.9736	0.0811

Figure 1: Performance results across thread counts

8 Discussion of Results

Observations:

- Maximum speedup occurs at 4 threads (S $\approx$ 1.6511).
- Speedup is greater than 1 for threads 2 through 9, indicating parallel execution is faster than sequential.
- Efficiency drops steadily as thread count increases: very high at 2 threads ($\approx$ 0.95), below 0.5 after 3 threads, and below 0.2 after 7 threads.
- Beyond 9 threads, speedup falls below 1 at 10–11 threads due to parallel overhead, then slightly recovers at 12 threads, likely because of OS scheduling or E-core involvement.

Observed Trend: Speedup is significant for small thread counts, reaching a maximum at 4 threads, and remains above 1 up to 9 threads. Beyond this, parallel overhead dominates and speedup falls below 1 for 10–11 threads, then slightly recovers at 12 threads. Efficiency steadily decreases with increasing thread count, dropping sharply after 4 threads.The observed final parameters (0.3103, 0.6149) reflect partial convergence on the scaled dataset. Scaling both features and targets drastically reduces gradient size, which avoids numerical instability (NaN/explosion) seen without scaling, but limits progress within only 120 epochs. This trade-off was intentionally chosen for stability on large N.

Key Limiting Factors:

1. Hybrid architecture: only 2 high-performance P-cores; E-cores are slower
2. Mutex contention during gradient accumulation (repeated every epoch)
3. Thread creation/join overhead per epoch
4. Shared L3 cache and memory bandwidth saturation
5. 15 W TDP $\rightarrow$ thermal/power throttling under sustained load

9 Conclusion:

The parallel implementation achieves meaningful speedup while respecting assignment constraints. Sub-linear scaling is expected on this low-power hybrid CPU and aligns with Amdahl's Law and practical multithreading realities.